lab08

Dados Relacionais com Polars

Objetivo

Dados relacionais são uma constante no exercício da profissão do estatístico. Esta estratégia permite uma representação mais efetiva de dados estruturados, oferecendo a possibilidade de análises computacionalmente mais eficientes.

Os Dados

O banco de dados para esta atividade é o "IMDb Movie Data", que possui informações sobre filmes, atores, diretores, gêneros e outros. Para os arquivos indicados abaixo, considere que o símbolo `\N` representa valores faltantes. Observe com cuidado a extensão dos arquivos para uma indicação do formato do mesmo.

1. title.basics0.tsv.gz (Informações Básicas dos Filmes) https://drive.google.com/file/d/1iYqAGTtIhRLK4ycFK41hYaWK3iKfOdT2/view?usp=sharing
2. title.ratings.tsv.gz (Avaliações dos Filmes) https://drive.google.com/file/d/1kZuj1lnXkPrNURzwYuc4FX
view?usp=sharing
3. title.principals0.tsv.gz (Elenco e Equipe Técnica) https://drive.google.com/file/d/1oLR2_mFyRHEiKlqFKt4QDVYlGH8LC4mn/view?usp=sharing

Atividade

Utilizando apenas operações por meio da biblioteca polars, responda:

```
library(reticulate)
py_require(c("numpy", "polars"))
```

```
import polars as pl
print(pl.__version__)
```

```
2.0.0
```

Exploração do banco de dados

```
basics = pl.read_csv("../title.basics0.tsv.gz",
                     separator = "\t",
                     null_values = "\\N")

print(basics.head())
```

```
shape: (5, 9)
```

tconst	titleType	primaryTitle	originalTitle	…	startYear	endYear	runtimeMinutes	genres
str	str	str	str		i64	str	i64	str
tt0000001	short	Carmencita	Carmencita	…	1894	null	1	Documentary,Short
tt0000002	short	Le clown et ses chiens	Le clown et ses chiens	…	1892	null	5	Animation,Short
tt0000003	short	Pauvre Pierrot	Pauvre Pierrot	…	1892	null	5	Animation,Comedy,Romance
tt0000004	short	Un bon bock	Un bon bock	…	1892	null	12	Animation,Short
tt0000005	short	Blacksmith Scene	Blacksmith Scene	…	1893	null	1	Comedy,Short

```
print(basics.shape)
```

```
(11144942, 9)
```

```
ratings = pl.read_csv("../title.ratings.tsv.gz",
                      separator = "\t",
                      null_values = "\\N")

print(ratings.head())
```

```
shape: (5, 3)

 tconst      averageRating   numVotes
 ---         ---             ---
 str         f64             i64

 tt0000001   5.7             2092
 tt0000002   5.6             283
 tt0000003   6.5             2099
 tt0000004   5.4             183
 tt0000005   6.2             2835
```

```
print(ratings.shape)
```

```
(1484615, 3)
```

```
principal = pl.read_csv("../title.principals0.tsv.gz",
                        separator = "\t",
                        null_values = "\\N")

print(principal.head())
```

```
shape: (5, 6)

 tconst      ordering   nconst      category          job                        characters
 ---         ---        ---         ---               ---                        ---
 str         i64        str         str               str                        str

 tt0000001   1          nm1588970   self              null                       [Self]
 tt0000001   2          nm0005690   director          null                       null
 tt0000001   3          nm0005690   producer          producer                   null
 tt0000001   4          nm0374658   cinematographer   director of photography   null
 tt0000002   1          nm0721526   director          null                       null
```

```
print(principal.shape)
```

```
(88359622, 6)
```

Quais são os 5 filmes com as maiores notas (averageRating)? Apresente uma solução capaz de desempatar os filmes baseando-se no número de votos recebidos.

```
filmes = ratings.join(basics,
                      on = "tconst",
                      how = "inner")

filmes = filmes.sort(
                    ["averageRating", "numVotes"],
                    descending = [True, True]
                    )

top5 = filmes.select(
                    ["originalTitle", "averageRating", "numVotes"]
                    ).head(5)

print(top5)
```

```
shape: (5, 3)

 originalTitle                    averageRating   numVotes
 ---                              ---             ---
 str                              f64             i64

 Ozymandias                       10.0            228900
 Kadifeyi Kesfet                  10.0            2904
 Tatilde CUK Oturur Mukemmel Du…  10.0            2878
 Inci Koleksiyonunu Kesfet!       10.0            2812
 DeFacto Kis Festivali            10.0            2676
```

Qual é o gênero mais frequente entre os filmes com nota maior que 8?

```
top_generos = (filmes
             .filter(pl.col("averageRating") > 8)
             .with_columns(pl.col("genres").str.split(","))
```

```
              .explode("genres")
              .group_by("genres")
              .agg(pl.len().alias("contagem"))
              .sort("contagem", descending = True)
              )

print(top_generos.head(1))
```

```
shape: (1, 2)

 genres   contagem
 ---      ---
 str      u32

 Drama    108356
```

Quais são os 3 atores/atrizes que mais participaram de filmes com nota maior que 7.5?

```
ratings_filtrados = (ratings
                     .filter(pl.col("averageRating") > 7.5)
                     .select("tconst", "averageRating")
                     )

#print(ratings_filtrados)

artistas = (principal
            .filter(pl.col("category").is_in(["actor", "actress"]))
            .select("tconst", "nconst")
            )

#print(artistas)


filmes_artistas = ratings_filtrados.join(artistas,
                                         on = "tconst",
                                         how = "inner")
#print(filmes_artistas.head(4))

top3_artistas = (
                 filmes_artistas
```

```
                .group_by("nconst")
                .agg(pl.col("tconst").n_unique().alias("quantidade_filmes"))
                .sort("quantidade_filmes", descending = True)
                .head(3)
                )
print(top3_artistas)
```

```
shape: (3, 2)

 nconst      quantidade_filmes
 ---         ---
 str         u32

 nm0048389   1901
 nm0217221   1676
 nm1477426   1416
```

```
Sys.setenv(TZ = "America/Sao_Paulo")

paste(
  "Arquivo compilado em:",
  format(Sys.time(), "%d/%m/%Y às %H:%M:%S")
)
```

```
[1] "Arquivo compilado em: 14/10/2026 às 09:32:35"
```